

Generative AI -- Complete Notes

Transformers · RAG · Training Pipeline · Production Architecture · Interview Guide

Spring Boot · Angular · Python · LangChain · OpenAI · Pinecone · May 2025

1. What is Generative AI?

Generative AI is a category of artificial intelligence model trained on massive amounts of existing content -- text, code, images -- that learns statistical patterns well enough to **generate new, original content**.

Generative AI = a model that generates new content by learning patterns from existing data at scale.

The three pillars

Pillar	What it is	Examples
Training data	What the model learned from	Books, code repos, web pages, conversations, scientific papers
LLM model	Compressed knowledge as billions of mathematical weights	GPT-4, Claude, Gemini, LLaMA
Application layer	What engineers build on top	Chatbots, code generators, RAG pipelines, AI agents

Traditional AI vs Generative AI

Traditional AI	Generative AI
Classifies or predicts from fixed categories	Generates new content -- text, code, images, audio
Rule-based or narrow supervised learning	Trained on internet-scale data via self-supervised learning
Example: spam classifier, fraud detector	Example: write an essay, generate code, answer any question

2. The Transformer Architecture

A Transformer is the neural network architecture powering every major LLM. Introduced in the 2017 paper "Attention is All You Need", it replaced RNNs by processing all tokens in parallel via self-attention.

How your prompt flows through a Transformer

```
Step 1 -- Tokenisation
"Hello world how are you" -> ["Hello", "world", "how", "are", "you"]
Each word (or sub-word) becomes a token with a unique integer ID.

Step 2 -- Embedding
Each token ID -> high-dimensional vector (e.g. 4,096 numbers for GPT-4)
Similar words have similar vectors. "king" - "man" + "woman" = "queen"

Step 3 -- Self-attention (repeated N times, e.g. 96 layers in GPT-4)
Every token looks at every other token and decides how much to attend to it.
Computed via Q (Query), K (Key), V (Value) matrices.

Step 4 -- Feed-forward network
Each token transformed independently using learned weights.

Step 5 -- Output (Softmax)
Final vector -> probability over entire vocabulary (~50,000 tokens)
Pick highest probability token -> that is the next word generated.
```

The Q, K, V mechanism -- the heart of attention

Role	Meaning	Example
Q -- Query	What am I looking for?	"bank" asking: am I near water or money?
K -- Key	What do I offer?	"river" advertising: I am a water feature
V -- Value	What content do I carry?	The actual information passed forward

The word "bank" in "river bank" attends strongly to "river" and weakly to financial words. Same word in "bank account" does the opposite. This context-awareness is computed mathematically.

Key vocabulary

Term	Plain English	Interview one-liner
Token	Smallest unit -- roughly a word or sub-word	1000 words = ~750 tokens; GPT-4 = ~4 chars per token
Embedding	Token converted to a vector of numbers	Similar meanings = similar vectors (measurable with cosine similarity)

Term	Plain English	Interview one-liner
Context window	Max tokens the model holds at once	GPT-4: 128K tokens = ~100,000 words
Temperature	Randomness control (0=deterministic, 2=chaotic)	Use 0 for factual Q&A, 0.7 for creative tasks
Hallucination	Confident but factually wrong output	Model fills gaps with plausible but incorrect information
Fine-tuning	Re-train a model on domain data	Changes model behaviour -- expensive but powerful
Parameters	Billions of numbers encoding model knowledge	GPT-4: ~1 trillion parameters

3. RAG -- Retrieval-Augmented Generation

LLMs only know what they were trained on. They cannot answer questions about your company's internal documents, events after their training cutoff, or real-time data. RAG solves this.

RAG grounds LLM answers in real documents at query time -- eliminating hallucination on domain knowledge.

Phase 1 -- Offline indexing (runs once)

```

Your documents (PDFs, databases, APIs)
|
Chunker -- split into ~500 token pieces
|
Embedding model -- convert each chunk to a vector
(e.g. OpenAI text-embedding-ada-002)
|
Vector database -- store all vectors
(Pinecone, Weaviate, pgvector, FAISS, Qdrant)

```

Phase 2 -- Online query (runs on every user question)

```

User question: "What is our refund policy?"
|
Embed the question using the same embedding model
|
Semantic search -- find top-K most similar chunks in vector DB
(cosine similarity between question vector and chunk vectors)
|
Build prompt:
  System: "Answer based only on the provided context."
  Context: [retrieved chunk 1] [retrieved chunk 2]
  Question: "What is our refund policy?"
  |
LLM generates grounded answer with citations
"Our refund policy allows 30 days... [source: policy.pdf p.4]"

```

RAG vs Fine-tuning

	RAG	Fine-tuning
Cost	Low -- no re-training	High -- GPU hours
Freshness	Real-time -- update vector DB	Stale -- must re-train to update
Transparency	Cites sources	Black box
Hallucination	Low -- grounded in documents	Can still hallucinate
Best for	Q&A; over documents, knowledge bases	Changing model style or behaviour

LangChain RAG -- Python example

```

rag_chain.py
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_pinecone import PineconeVectorStore

# Build the RAG chain
vectorstore = PineconeVectorStore(index_name="my-index",
                                   embedding=OpenAIEmbeddings())
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4o", temperature=0.1),
    retriever=vectorstore.as_retriever(search_kwargs={"k": 4}),
    return_source_documents=True
)

# Answer a question
result = qa_chain.invoke({"query": "What is our refund policy?"})
print(result["result"])
# Sources:
for doc in result["source_documents"]:
    print(doc.metadata["source"])

```

4. LLM Training Pipeline

Every major LLM -- GPT-4, Claude, Gemini, LLaMA -- goes through the same four-stage training pipeline.

Stage 1 -- Pre-training

- Input: Trillions of tokens from the internet, books, code, scientific papers
- Task: Next-token prediction -- given previous tokens, predict the next one
- Scale: Months of training on thousands of GPUs
- Output: Base model that understands language but does not follow instructions

Stage 2 -- Supervised Fine-Tuning (SFT)

- Input: Human-written instruction + answer pairs (thousands to millions of examples)
- Task: Learn to follow instructions in a helpful, structured format
- Scale: Days on fewer GPUs
- Output: A model that responds to prompts in a structured, helpful way

Stage 3 -- RLHF (Reinforcement Learning from Human Feedback)

1. Generate multiple responses to the same prompt
2. Human raters rank the responses from best to worst
3. Train a reward model to predict human preferences
4. PPO optimisation -- update the LLM to score higher on the reward model

Output: Helpful, harmless, honest model
(Anthropic's Constitutional AI extends this further)

Stage 4 -- Deployment

- Quantisation -- compress from float32 to int8/4-bit (reduces size 4-8x)
 - API serving -- vLLM, TensorRT, or managed endpoints (OpenAI, Anthropic, Azure)
 - Rate limiting -- tokens per minute, requests per day
 - Guardrails -- content moderation, prompt injection defence
 - Monitoring -- latency, token cost, error rates, hallucination detection
-

5. Production Gen AI Architecture

```

User Interface (Angular / React)
  streams tokens via SSE, renders citations
  |
API Gateway (Spring Boot / FastAPI)
  Auth, rate limiting, request validation
  |
Orchestration Layer (LangChain / LlamaIndex)
  retrieve -> build prompt -> call LLM -> parse -> return with citations
  Also: agents, tool calling, memory, multi-step chains
  |
+-----+-----+-----+
|               |               |               |
Vector DB       LLM Provider     Memory Store
Pinecone /      OpenAI /         Redis /
pgvector        Anthropic /      DynamoDB
Semantic        Azure OAI        Session context
search                                     between turns
|
Observability + Safety
Token cost, latency, hallucination detection,
prompt injection defence, A/B testing, drift detection

```

Spring Boot + Spring AI integration

```

GenAIService.java
@Service
public class GenAIService {
    private final OpenAiChatClient chatClient;
    private final VectorStore vectorStore;

    public String ragAnswer(String userQuestion) {
        // Semantic search in vector store
        List<Document> docs =
            vectorStore.similaritySearch(userQuestion, 4);
        // Build context from retrieved docs
        String context = docs.stream()
            .map(Document::getContent)
            .collect(Collectors.joining());
        // Build prompt and call LLM
        String prompt = context + userQuestion;
        return chatClient.call(new Prompt(prompt))
            .getResult().getOutput().getContent();
    }
}

```

Python FastAPI + LangChain (KraftHeinz-style)

```

main.py
from fastapi import FastAPI
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_pinecone import PineconeVectorStore
from langchain.chains import RetrievalQA

app = FastAPI()

embeddings = OpenAIEmbeddings(model="text-embedding-ada-002")
vectorstore = PineconeVectorStore(index_name="compass-docs",
                                   embedding=embeddings)
llm = ChatOpenAI(model="gpt-4o", temperature=0.1, streaming=True)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(search_kwargs={"k": 4}),
    return_source_documents=True
)

@app.post("/api/ai/query")
async def query(question: str):
    result = qa_chain.invoke({"query": question})
    return {
        "answer": result["result"],
        "sources": [doc.metadata["source"]
                    for doc in result["source_documents"]]
    }

```

6. Key Concepts Quick Reference

Core vocabulary table

Term	Definition	Interview one-liner
LLM	Large Language Model -- transformer trained on billions of tokens	"A statistical model predicting the most likely next token given context"
RAG	Retrieve docs at query time, inject into prompt	"Grounds LLM in real documents, eliminates domain hallucination"
Embedding	Token converted to a high-dimensional vector	"Similar meanings = similar vectors -- measurable with cosine similarity"
Attention	Every token looks at every other token simultaneously	"Captures context: bank means different things near river vs money"
RLHF	Humans rank outputs to train a reward model	"Makes LLM helpful, harmless, honest via PPO optimisation"
Hallucination	Confident but factually wrong output	"Model fills knowledge gaps with plausible but incorrect information"

Term	Definition	Interview one-liner
Fine-tuning	Re-train a model on domain-specific data	"Changes model behaviour -- expensive but powerful for domain adaptation"
Temperature	Randomness control 0-2	"0 for factual Q&A, 0.7 for creative tasks, 0 for production RAG"
Vector DB	Stores embeddings, enables semantic search	"Query by meaning not keywords -- pgvector, Pinecone, Weaviate"
Context window	Max tokens the model processes at once	"GPT-4: 128K tokens; Claude: 200K tokens"

Prompt engineering patterns

```
prompt_patterns.py
# Pattern 1: System + User structure
messages = [
    {'role': 'system', 'content': 'You are a Java expert. Be concise.'},
    {'role': 'user', 'content': 'Explain Optional<T> in Java 8'}
]

# Pattern 2: Few-shot examples
prompt = 'Informal: gonna fix it -> Formal: will be resolved.'

# Pattern 3: Chain-of-thought
prompt = 'Solve step by step: If a train leaves at 9am...'

# Pattern 4: Structured JSON output
prompt = 'Return ONLY JSON: {issues:[...], severity: low|medium|high}'
```

7. Interview Answer Guide

Q: What is Generative AI and how does it work?

Generative AI is a model trained on massive datasets to learn statistical patterns and generate new content. The key architecture is the Transformer -- it converts text into token embeddings, passes them through many self-attention layers where every token attends to every other token, and predicts the next most likely token. Modern LLMs are pre-trained on trillions of tokens, then fine-tuned with RLHF to be helpful and safe. For enterprise applications, the RAG pattern grounds answers in real documents and eliminates hallucination.

Q: What is RAG and when would you use it?

RAG stands for Retrieval-Augmented Generation. It solves the core problem that LLMs only know their training data. In the offline phase you chunk documents, embed each chunk as a vector, and store in a vector database. At query time you embed the question, semantically search for relevant chunks, inject them into the prompt, and the model generates a grounded answer with citations. At KraftHeinz I built a RAG pipeline letting sales teams query Compass policy documents in natural language, with answers backed by specific document sources.

Q: What is hallucination and how do you prevent it?

Hallucination is when a model generates confident but factually incorrect output -- filling knowledge gaps with statistically plausible but wrong information. The three main mitigations are: RAG (ground answers in real documents), temperature 0 for factual queries (forces most likely token), and output validation (check responses against known facts or require citations). Monitoring response quality via LLM-as-judge patterns catches drift in production.

Q: How would you integrate an LLM into a Spring Boot application?

I would use Spring AI for consistent abstraction over OpenAI, Anthropic, and Azure OpenAI. Angular frontend streams tokens via SSE. Spring Boot API gateway handles auth and rate limiting. LangChain4j or Spring AI orchestrates the RAG chain -- semantic search in pgvector, prompt construction, LLM call, response parsing. Redis stores session context for conversation memory. CloudWatch monitors token usage and latency. I implemented exactly this at KraftHeinz -- a FastAPI Python service for AI-powered data extraction integrated with the Spring Boot backend.